Intro2cs Fall 2021-2 Moed Bet Exam

Exam instructions:

<u>מועד ב, חלק ראשון (שעת התחלה: 13:30)</u>

סיסמת הכניסה לבחינה היא: 123
הסיסמא תהיה פעילה **רק החל משעה 13:30** (לחלק הראשון) **ו- 16:00** (לחלק השני).
לפני כן סיסמה זו תתן הודעות שגיאה בנסיונות התחברות.

<u>קראו בזהירות את ההנחיות!</u>

במבחן זה שני חלקים. לכל חלק מוקצות 90 דקות (לבעלי הארכת זמן תינתן תוספת בהתאם). אין לצאת לשירותים במהלך הבחינה, אלא רק בהפסקה בין שני החלקים. הבחינה תתקיים בהשגחה בזום ודרך מערכת SEB.

<u>מבנה הבחינה:</u>
בכל חלק יש

- **5 שאלות קצרות** (4 נק כל אחת). **יש לענות על <u>כל</u> השאלות הקצרות.**
- **3 שאלות תכנות** (15 נק כל אחת). **יש לענות רק על 2 מתוך ה- 3**. בסוף כל חלק תתבקשו לסמן אילו שאלות מה-3 בחרתם.

<u>הנחיות נוספות:</u>
- בשאלות התכנות עליכם לכתוב קוד יעיל ונקי ככל האפשר. תוכניות לא יעילות או שאינן כתובות היטב לא יקבלו את כל הנקודות גם אם הן עובדות נכון.
- בכל שאלות התכנות מותר לכתוב פונקציות עזר נוספות ללא הגבלה.
- אין צורך לתעד את הקוד. אין צורך ב-docstring. הערות לקוד במקומות בהם כתבתם משהו מורכב עשויות לסייע בבדיקת התשובה שלכם אך אינן הכרחיות.
- **הבחינה היא עם חומר סגור. אין לעשות שימוש במחשבון או בדפי נוסחאות, או בכל חומר עזר אחר.**
- מותר לעשות שימוש בדפי טיוטא חלקים. אין צורך להגישם.

<u>שאלות לצוות הקורס</u>
- **שאלות יש להפנות דרך הצ'אט של מודל בלבד**.
- מענה עשוי לקחת כמה דקות בגלל העומס. אם לא נעניתם אל תכתבו הודעות נוספות שיעכבו את המענה עוד יותר אלא המתינו בסבלנות.
- אנא ציינו בכל פנייה את מספר השאלה עליה אתם שואלים.
- אנא המנעו משאלות שאינן קשורות ישירות להבנת השאלה. צוות הקורס לא יוכל לספק רמזים או לענות על שאלות אחרות במהלך המבחן.
- אין צורך לענות לנו בהודעות "תודה" לאחר שהשבנו על שאלות. זה ממלא את מסכי הצ'אט ויגרום לעיכובים נוספים במענה לשאלות.

בהצלחה!

Part 1

Q1 had a textbox with an interpreter to allow students to run code. It isn't checked.

Part 1, Q2

נתונה תוכנית הבונה GUI עם 3 כפתורים:

```
import tkinter as tki

top = tki.Tk()
buttons = [tki.Button(top, text="Button" + str(i)) for i in range(3)]
for button in buttons:
    button.pack()
```

הGUI שבונה התוכנית נראה כך:



המשך התוכנית קובע לכל כפתור פעולה באופן הבא:

```
def f(t): return lambda: print(t, end="")

def g(x, y):
    count = [False]
    def foo():
        count[0] = not count[0]
        x() if count[0] else y()
    return foo

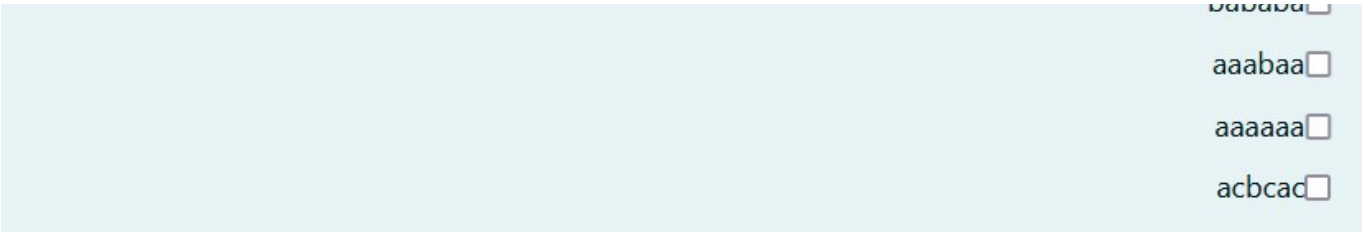
x = g(f("a"), f("b"))
buttons[0]["command"] = f("a")
buttons[1]["command"] = x
buttons[2]["command"] = x
g = g(x, f("c"))

top.mainloop()  # start up the GUI
```

לאחר הרצת התוכנית יוסף החל ללחוץ על הכפתורים באקראי.

אילו מהמחרוזות הבאות יכולות להיות מודפסות? (סמנו את כל האפשרויות הנכונות, ורק אותן.)

- ☐ aabbab
- ☐ aabaab
- ☐ bababa



Possible strings are only:

- aabaab
- aaabaa
- aaaaaa

(b can't appear twice in a row, it can't be the first character in the string, c will never appear).

Part 1, Q3

נתונה התכנית הבאה:

```
from functools import reduce
from typing import List

print_message = True
msg = "the product is:" if print_message else ""

lst: List[int] = [1, -2, 0, 4, -5, 6, 0, 8, 9]

g = ???

print(msg, reduce(g, lst))
```

מה צריך להיות הערך של `g` כדי שבשורה האחרונה תודפס **מכפלתם של כל המספרים שאינם 0** ברשימה `lst`? פתרונכם צריך לעבוד לכל רשימה של מספרים שלמים, ולא רק לזו שנתונה בדוגמא. עבור הדוגמא הנתונה יודפס:

```
the product is:  17280
```

שהרי: 1* (-2) * 4 * (-5) * 6 * 8 * 9 = 17280

כתבו כאן רק את מה שמחליף את סימני השאלה בקוד למעלה:

לנוחיותכם, הנה התיעוד של פונקציית reduce:

```
Help on built-in function reduce in module _functools:

reduce(...)
    reduce(function, sequence[, initial]) -> value

    Apply a function of two arguments cumulatively to the items of a sequence,
    from left to right, so as to reduce the sequence to a single value.
    For example, reduce(lambda x, y: x+y, [1, 2, 3, 4, 5]) calculates
    ((((1+2)+3)+4)+5).  If initial is present, it is placed before the items
    of the sequence in the calculation, and serves as a default when the
    sequence is empty.
```

```
g = lambda a,x: (a if a else 1) * (x if x else 1)
```

Part 1, Q4

נתונה הפונקציה הבאה:

```
def f(n):
    result = 0
    result2 = []
    for i in range(n):
        j = n
        while j > 10:
            result += j*i
            j = j//3

    for i in range(n):
        result2.append(i*n)

    return result
```

מהי סיבוכיות זמן הריצה של `f` כפונקציה של `n` במקרה הגרוע ביותר? ציינו את החסם ההדוק ביותר.

$O(n \cdot log(n))$

Part 1, Q5

נתונה הפונקציה הבאה:

```
def f(n):
    res = []
    j = 1
    for i in range(5*n):
        lst = [i%5 for i in range(i,100*n+i)]
        if i>j:
            res.append(lst)
            j = j + j
    return res
```

מהי סיבוכיות המקום של `f` (כלומר כמה זכרון דרוש להרצתה) כפונקציה של `n` במקרה הגרוע ביותר? ציינו את החסם ההדוק ביותר.

$O(n \cdot log(n))$

Part 1, Q6

נתונה התכנית הבאה:

```
def f(n,m):
    x = g(n) + g(m)
    return x+2

def g(j):
    if j<=0:
        return 1
    else:
        return f(j-1,j-2) + f(j-2,j-3)

g(k)
```

שימו לב שבשורה האחרונה קוראים לפונקציה g עם פרמטר k. הניחו ש-k מספר שלם חיובי.

מהו העומק המקסימלי של מחסנית הקריאות בזמן הרצת התוכנית? במילים אחרות, כמה עמוקה היא הרקורסיה (כולל גם קריאות לפונקציה g, וגם לפונקציה f)? כתבו את תשובתכם כביטוי התלוי בפרמטר k.

The maximal depth is $2k+1$

Part 1, Q7

כתבו פונקציה `get_k_tuples(it, k)` שמקבלת איטרטור `it` ומספר שלם חיובי `k` ומחזירה איטרטור.

האיטרטור שחוזר יוציא את האיברים שהיו באיטרטור המקורי בטאפלים בגודל `k`. הטאפלים יכילו איברים עוקבים מהאיטרטור המקורי כשכל טאפל מתחיל באיבר עוקב לאיבר הראשון בטאפל הקודם. לדוגמא, אם האיטרטור `it` החזיר את האיברים `1,2,3,4,5` (לפי סדר עולה משמאל לימין) אזי האיטרטור שמחזירה הקריאה `get_k_tuples(it, 2)` יוציא את הטאפלים

```
(1, 2) (2, 3) (3, 4) (4, 5)
```

שימו לב שמוחזרים פחות טאפלים ממספר האיברים באיטרטור `it` המקורי. בפרט, אם `k` גדול ממספר האיברים באיטרטור `it`, אזי האיטרטור שמחזירה הפונקציה יהיה איטרטור ריק.

<u>דוגמאות קוד נוספות:</u>

```
lst = [1, 2, 3, 4, 5]
print(*get_k_tuples(iter(lst), 3), end="")
# prints (1, 2, 3) (2, 3, 4) (3, 4, 5)

print(*get_k_tuples(iter(lst), 6), end="")
# prints nothing.
```

<u>הערות:</u>

- כדי לממש את האיטרטור מותר לעשות שימוש בגנרטור או לייצר מחלקה עבורו (לבחירתכם).
- **אין לעשות שימוש במודול itertools.**
- האיטרטור חייב להיות **חסכוני בזכרון** ויעיל ככל הניתן.
- שימו לב שהפונקציה מקבלת איטרטור (ולא iterable).
- ניתן להניח ש-k הוא מספר חיובי.

In [1]:
```
def get_k_tuples(it, k):
    lst = []
    try:
        for i in range(k):
                lst.append(next(it))
        yield tuple(lst)
        while True:
            del lst[0]
            lst.append(next(it))
            yield tuple(lst)
    except StopIteration:
        return
```

Part 1, Q8

נתונה המחלקה Node המייצגת קודקוד ברשימה מקושרת. אין לשנותה. יש לעשות בה שימוש כפי שהיא ואין צורך להוסיף לה מתודות getters או setters.

```
class Node:
    def __init__(self, data, next = None):
        self.data, self.next = data, next
```

כתבו פונקציה

```
find_max_sum_pair(head: Node)->float
```

מלבד הרשימה שכבר נמצאת בזכרון, הפונקציה יכולה להשתמש רק **בכמות קבועה של זכרון נוסף** בעת ריצתה (ראו הערות בסוף השאלה). היא מקבלת מצביע לראש של רשימה מקושרת המכילה מספרים (כלומר נתונים מטיפוסים int או float). עליה לבדוק את הסכום של זוגות איברים ברשימה המקושרת ולהחזיר את הסכום המקסימלי שנמצא. הזוגות שיש לבדוק הם זוגות במרחק שווה מאמצע הרשימה. כלומר, בודקים את:

- האיבר הראשון ברשימה + האיבר האחרון ברשימה.
- האיבר השני ברשימה + האיבר השני מהסוף ברשימה.
- האיבר השלישי ברשימה + האיבר השלישי מהסוף ברשימה.
- וכן הלאה...

אם הרשימה באורך אי-זוגי, האיבר באמצע הרשימה נסכם עם עצמו.

לדוגמא, עבור הרשימה המקושרת המכילה את האיברים

(7) --> (1) --> (2) --> (4) --> (-3) --> (-2) -->

יש להחזיר את הערך 6 כי:

- 7+(-2) = 5
- 1+(-3) = -2
- 2+4 = 6 וזהו הערך המקסימלי

<u>שימו לב:</u>

- **<u>על הפתרון שלכם לעבוד בסיבוכיות מקום O(1).</u>** אין להקצות זכרון נוסף התלוי באורך הקלט. **פתרונות בסיבוכיות מקום גבוהה לא יקבלו יותר מ-5 נקודות.** (לצורך שאלה זו, מספר שלם (int) יחיד מאפשר למנות את מספר האיברים ברשימה ונחשב לזכרון בגודל קבוע).
- **<u>אסור לעשות שום שינוי ברשימה המקושרת הנתונה.</u>**
- מותר להניח שברשימה המקושרת יש לפחות איבר אחד.

This question cannot be solved in less that $O(n^2)$ running time. It is useful to have the length of the list, and a function to fetch the item at index i.

In [2]:
```
class Node:
    def __init__(self, data, next=None):
        self.data, self.next = data, next

# The code above was provided.

def get_length(head):
    res = 0
    while head:
        res += 1
        head = head.next
    return res


def fetch(head, ind):
    for i in range(ind):
        head = head.next
    return head.data


def find_max_sum_pair(head: Node) -> float:
    n = get_length(head)
    max_sum = head.data + fetch(head, n-1)
    for i in range(1, (n+1)//2):
        head = head.next
        max_sum = max(max_sum, head.data + fetch(head, n-2*i-1))
    return max_sum
```

Part 1, Q9

בשאלה זו אסור ליבא מודולים חיצוניים כגון itertools, functools

כתבו פונקציה `xyz(num_x, num_y, num_z)` המקבלת שלושה מספרים שלמים num_x, num_y, num_z. הפונקציה מחזירה רשימה המכילה את כל המחרוזות השונות בהן יש בדיוק:

- num_x פעמים את התו "x",
- num_y פעמים את התו "y",
- num_z פעמים את התו "z".

אין לכלול במחרוזות אף תו אחר.

לדוגמא, הקריאה xyz(2, 0, 1) תחזיר את הרשימה: ['xxz', 'xzx', 'zxx']

בהינתן ההגדרה הנ"ל הקריאה xyz(0, 0, 0) צריכה להחזיר רשימה המכילה מחרוזת ריקה (זו המחרוזת היחידה בה יש 0 מופעים של התווים x,y,z ואין בה אף תו אחר).

הערות נוספות:

- הניחו כי num_x, num_y, num_z הם אי שליליים (כלומר הם או 0 או חיוביים).
- אין חשיבות לסדר החזרת המחרוזות בתוך הרשימה.
- כל מחרוזת מתאימה תופיע ברשימה רק פעם אחת.
- מותר (כמו תמיד) לעשות שימוש בפונקציות עזר.

In [3]:
```
from typing import List
def xyz(num_x:int, num_y:int, num_z:int) -> List[str]:
    result = []
    helper(num_x, num_y, num_z, result, "")
    return result


def helper(x, y, z, result, initial_str):
    if x == 0 and y == 0 and z == 0:
        result.append(initial_str)
        return
    if x > 0:
        helper(x-1, y, z, result, initial_str+"x")
    if y > 0:
        helper(x, y-1, z, result, initial_str+"y")
    if z > 0:
        helper(x, y, z-1, result, initial_str+"z")
```

Part 2

Q1 had a textbox with an interpreter to allow students to run code. It isn't checked.

Part 2, Q2

נתונה התכנית הבאה:

```
x = [[[0]*4]*4]*4

z = [1, 2, 3]
z.append(z)
z.append(z*2)
```

סעיף א: לאחר הרצתה, לכמה רשימות מצביע המשתנה x באופן ישיר או עקיף? סיפרו גם את הרשימה שב-x וגם כל רשימה שמוכלת בה כל עוד הן מהוות אובייקטים שונים ממש בזכרון.

סעיף ב': לכמה רשימות מצביע המשתנה z באופן ישיר או עקיף? סיפרו גם את הרשימה שב-z וגם כל רשימה שמוכלת בה כל עוד הן מהוות אובייקטים שונים ממש בזכרון.

x contains only 3 lists.

z contains only 2 lists.

Part 2, Q3

נתון מילון d שממפה מספרים שלמים (int) למספרים שלמים (int).

מה צריך לכתוב במקום סימני השאלה בתוכנית הבאה כדי שהפונקציה invert_dictionary תהפוך את המילון? (שימו לב לדוגמת הקוד שלמטה).

במילון ההפוך, כל ערך של המילון המקורי הופך למפתח. היות ויכולים להיות מספר מפתחות במילון המקורי הממפים לאותו ערך, לא נמפה אליהם ישירות. במקום זאת, כל מפתח במילון המקורי יתווסף לרשימה אליה ממפים הערכים התואמים. למשל, אם במילון המקורי המפתח 7 הותאם לערך 1, אזי במילון ההפוך המספר 1 ימופה **לרשימה** שתכיל את 7.

```
d = {7:1, 8:2, 9:3, 10:3, 11:2, 12:2 }

def invert_dictionary(d):
    return ???

print(invert_dictionary(d))
# prints:
# {1: [7], 2: [8, 11, 12], 3: [9, 10]}
```

על תוכניתכם לעבוד עבור כל מילון הממפה מספרים שלמים למספרים שלמים ולא רק עבור הדוגמא הנתונה.

כתבו כאן רק את מה שמחליף את סימני השאלה בקוד למעלה:

```
{v:[k for k in d if d[k]==v] for v in d.values()}
```

Part 2, Q4

נתונה התכנית הבאה:

```
def f(x:int) -> ???:
    if x < 0:
        return
    else:
        return x

g = {2: [1,"a"], 7: [3, "the", 4, "clip"]}
```

סעיף א: מהו הטיפוס של ערך ההחזרה של f? (כלומר, מה יש לרשום במקום סימני השאלה?) רישמו את הטיפוס המדוייק ביותר שאפשר לתת בפורמט שיזוהה ע"י mypy.

סעיף ב: מהו הטיפוס של המשתנה g? רישמו את הטיפוס המדוייק ביותר שאפשר לתת בפורמט שיזוהה ע"י mypy.

- Optional[int]
- Dict[int, List[Union[int,str]]]

Part 2, Q5

נתונה התוכנית הבאה:

```
def g(lst, frm, to):
    mid = (frm+to)//2
    if frm >= to:
        return 1
    else:
        z = lst[mid] % 3
        return (g(lst,frm+1,to) + g(lst,frm,to-1) + z)/2


lst = list(range(n))
g(lst, 0, n)
```

מהי סיבוכיות זמן הריצה של התוכנית כפונקציה של `n` במקרה הגרוע ביותר? ציינו את החסם ההדוק ביותר.

$O(2^n)$

Part 2, Q6

נתונה המחלקה הבאה המייצגת קודקוד בעץ:

```
class Node:
    def __init__(self, data, children = None):
        self.data = data
        if not children:
            self.children = []
        else:
            self.children = children[:]
```

נתונה תוכנית העושה שימוש במחלקה זו:

```
def next_level(nodes):
    for node in nodes:
        ???

def print_depth_2(root):
    for n in next_level(next_level([root])):
        print(n.data)
```

מה יש לרשום במקום סימני השאלה (???) כדי שהפונקציה `print_depth_2(root)` תדפיס את הנתונים המצויים בנכדיו של שורש העץ שמועבר אליה כפרמטר? (כלומר כדי שיודפסו הנתונים בקודקודים בעומק 2 בעץ כשעומק השורש הוא-0).

```
yield from node.children
```

Part 2, Q7

כתבו פונקציה

```
intersect(intervals: List[Tuple[float,float], points: List[float]) -> bool
```

הפונקציה מקבלת רשימה `intervals` ורשימה `points`.

- הרשימה `intervals` מכילה טאפלים באורך 2. כל טאפל `(x,y)` מייצג את הקטע **הסגור** על הישר מהנקודה `x` אל הנקודה `y` (כלומר הקטע כולל את הנקודות בקצוות).
- הרשימה `points` היא רשימה של נקודות על הישר.

הפונקציה מחזירה `True` אם ורק אם אחת (או יותר) מהנקודות ב-`points` מוכלת באחד (או יותר) מהקטעים הסגורים ב- `intervals`. אחרת הפונקציה מחזירה `False`.

לדוגמא:

- `intersect([(3.5, 4.5), (-1,2), (3,4)], [2.5, 5.5])  #returns False`
- `intersect([(1,2), (3,4)], [1])  #returns True`
- `intersect([(1,2), (3,4)], [7, 3.5]) #returns True`

<u>דגשים והערות:</u>

- **הפתרון שלכם חייב להיות יעיל ככל הניתן**. פתרונות בסיבוכיות לא מיטבית יאבדו נקודות.
- כל הנקודות והקואורדינטות הן מספרים (מטיפוס float או int).
- כל קטע סגור הנתון בטאפל `(x,y)` מקיים `x<=y`.
- מותר לשנות את הרשימות שניתנו כקלט לפונקציה.
- רשימת האינטרוולים עשוייה להכיל אינטרוולים חופפים. רשימת הנקודות יכולה להכיל את אותה הנקודה כמה פעמים.
- **כל אחת משתי הרשימות עשויה להיות ריקה.** במקרה זה אף נקודה לא מוכלת באף קטע ולכן יש להחזיר `False`.

In [4]:
```python
from typing import List, Tuple

def intersect(intervals: List[Tuple[float,float]], points: List[float])
-> bool:
    intervals.sort(key = lambda x:x[1])
    points.sort()
    while True:
        if not points or not intervals:
            return False
        p = points.pop()
        s,e = intervals.pop()
        if p>e:
            intervals.append((s,e))
        elif p<s:
            points.append(p)
        else:
            return True
```

Part 2, Q8

נזכור שבעץ מגדירים את העומק של כל קודקוד כמספר הצעדים שיש לעשות כדי להגיע אליו מהשורש.
(השורש עצמו הוא בעומק 0, ילדיו בעומק 1 וכן הלאה.)

נתונה המחלקה `Node` הממשת עץ (אין לשנותה. מותר לגשת ישירות לשדות שלה, ואין צורך במתודות get או set).

```python
class Node:
    def __init__(self, data, children):
        self.data, self.children = data, children
```

כתבו פונקציה `check_tree(root)` המקבלת שורש של עץ, ובודקת אם כל העלים שבעץ הם בעומק זוגי, אי זוגי, או גם וגם.

הפונקציה תחזיר אחד מהערכים **0, 1, או None.**

- אם כל עלי העץ הם בעומק זוגי: **יוחזר 0**
- אם כל עלי העץ הם בעומק אי-זוגי: **יוחזר 1**
- אם יש בעץ גם עלים בעומק זוגי וגם עלים בעומק אי זוגי, **יש להחזיר None**

<u>הנחות על הקלט:</u>

- `root` הוא מטיפוס `Node` ובפרט אינו `None`. מבנה העץ תקין (אין מעגלים וכל ההפניות הן לאובייקטים מטיפוס `Node`, כמקובל בעצים).
- במחלקה `Node`, השדה `children` מכיל רשימה של קודקודים שהם ילדיו של הקודקוד הנוכחי (יכול להיות מספר ילדים כלשהו).
- לעלה יש רשימת ילדים שהיא רשימה ריקה (ולכן אין בעץ הפניות ל-`None`).

In [5]:

```python
from typing import Optional

class Node:
    def __init__(self, data, children):
        self.data, self.children = data, children

# The code above was provided:

def check_tree(root: Node) -> Optional[int]:
    if not root.children:
        return 0
    l = check_tree(root.children[0])
    if l == None:
        return None
    for i in range(1, len(root.children)):
        r = check_tree(root.children[i])
        if r == None or l != r:
            return None
    return 1-l
```

Part 2, Q9

אין צורך לטפל ב-keyword arguments בשאלה זו. כלומר, לפונקציה המקושטת לא יקראו עם keywords.

נזכור כי פונקציה f הנתונה בצורת curry היא פונקציה המקבלת את הפרמטרים שלה אחד אחד. למשל:
f(1)(2)(3)

כתבו פונקציה בשם `uncurry` כך שכשמקשטים פונקציה f בעזרת הדקורטור `uncurry(n)` היא הופכת מפונקציה בצורת curry לפונקציה "רגילה" המקבלת את הפרמטרים שלה בבת אחת. הפרמטר `n` מציין את מספר הפרמטרים שיש לקבל. אם ניתן לפונקציה המקושטת מספר פרמטרים שונה מ-`n`, יש לזרוק שגיאה מסוג `TypeError` (שמוגדרת כבר בפייתון). ניתן להניח ש-`n` מספר שלם חיובי ממש.

<u>לדוגמא:</u>

הנה פונקציה לא מקושטת:

```
def f(x):
    return lambda y: lambda z: x+y+z

f(1)(2)(3)   #returns 6
```

כך מתנהגת פונקציה מקושטת:

```
@uncurry(3)
def f(x):
    return lambda y: lambda z: x+y+z

f(1,2,3)  #returns 6
f(1,2)    #raises a TypeError exception
```

הערות:

- אין לעשות שימוש במודול `functools` בעת פתרון הבעיה. אין צורך לעשות שימוש ב- `@wraps`.
- ניתן להניח ש-`n` מספר שלם חיובי ממש.

In [6]:
```
def uncurry(n):
    def dec(f):
        def inner(*args):
            if len(args) !=n:
                raise TypeError("wrong number of arguments given", len
(args))
            res = f
            for arg in args:
                res = res(arg)
            return res
        return inner
    return dec
```